Artificial Intelligence 1DL340
A Short Tutorial for R

Amin Kaveh

September 2021

1 Data Types and Data Structures

R has 5 basic data types.

- character (e.g., "Artificial Intelligence")
- numeric (real or decimal)
- integer
- logical (i.e., TRUE, FALSE)
- complex (e.g., 2i+3)

The aforementioned data types can be structured in the following data structures:

- vector
- matrix
- dataframe
- list

1.1 vectors

A vector is a collection of elements of the same basic data types such as character, logical, integer or numeric. To create a vector, command c() which stands for *concatenate* is used:

```
> myNumbers <- c(1,2,3)
> myCourses <- c("Artificial Intelligence","Machine Learning","Data Mining")
```

As you can see we have not defined the data type and R does that based on the elements that we have given to it as input. To see the type of data in a vector, we can use typeof():

```
> typeof(myNumbers)
[1] "double"
> typeof(myCourses)
[1] "character"
```

To define an empty vector with a specific data type and length we can usse the following command:

```
> myFinalVector <- vector(mode = "logical",length = 10)
```

1.2 Matrix

A matrix in R is simply a two-dimensions vector. Therefore, similar to vectors, elements in a matrix has to be of the same basic data type:

```
> A <- matrix(1:16, ncol = 4, nrow = 4)
> A
     [,1] [,2] [,3] [,4]
[1,]    1    5    9   13
[2,]    2    6   10   14
[3,]    3    7   11   15
[4,]    4    8   12   16
```

Please pay attention that R is a column-wise language, means that it fills columns consecutively with input elements. If you want to insert data into a matrix row-wise, you can use the `byrow` switch.

```
> A <- matrix(1:16, ncol = 4, nrow = 4, byrow = TRUE)
> A
     [,1] [,2] [,3] [,4]
[1,]    1    2    3    4
[2,]    5    6    7    8
[3,]    9   10   11   12
[4,]   13   14   15   16
```

Matrix Manipulation

In this section, we introduce some functions that are helpful to work with matrices in R:

```
# it doubles the values in rows 2 and 3
> A <- A[2:3,] * 2

# it doubles the values in columns 1 and 3
> A <- A[,c(1,3)] * 2

# dimension of a matrix
> dim(A)
[1] 4 4

# transpose of a matrix
> transposeA <- t(A)

# multiplying two matrices:
matrix1 <- matrix(1:6, nrow = 2, ncol = 3)
matrix2 <- matrix(11:16, nrow = 3, ncol = 2)

# matrix1 %*% matrix2 yields a 2 in 2 matrix
> matrix1 %*% matrix2
     [,1] [,2]
```

```
[1,]  112  139
[2,]  148  184

# to multiply element-wise two matrices with the same dimensions we use *
    ↪ without
> matrix1 * t(matrix2)
     [,1] [,2] [,3]
[1,]   11   36   65
[2,]   28   60   96

# to find the max value of a matrix
> max(matrix2)
[1] 16

# to find the address of the max value in a matrix
> which(matrix2 == max(matrix2), arr.ind = TRUE)
    row col
[1,]   3   2

# to use the address we can assign the output of the previous command to a
    ↪ variable and use that variable as a vector
> maxIndices <- which(matrix2 == max(matrix2), arr.ind = TRUE)

> maxIndices[1]
[1] 3

> maxIndices[2]
[1] 2

# to append a new row at the end of a matrix (rbind)
> matrix1 <- rbind(matrix1, c(10,11,12))
> matrix1
     [,1] [,2] [,3]
[1,]    1    3    5
[2,]    2    4    6
[3,]   10   11   12

# to append a new column at the end of a matrix (cbind)
> matrix2 <- cbind(matrix2, c(1,2,3))
> matrix2
     [,1] [,2] [,3]
[1,]   11   14    1
[2,]   12   15    2
[3,]   13   16    3
```

1.3 dataframe

The `vector` and `matrix` data structures are for the situations that the data type of all elements are the same. Of course it not the regular case when we are working with the real world's data. The two data structures that can keep heterogeneous data types are `dataframe` and`list`. We can look at a data frame as a combination of multiple `vectors` with the same size but not necessarily the same type.

```
# To know the projects' deadlines and precision requirements please read the
```

```
    ↪ course page on studium!
> projectNames <- c("DeliveryMan", "WheresCroc", "Diagnostics")
> projDeadlines <- as.Date(c("2021-09-30", "2021-09-30", "2021-10-15"),format =
    ↪  "%Y-%m-%d")
> precisionRequirements <- c(180, 8, 0.99)

# To construct a dataframe
projectsAI <- data.frame(projectNames, projDeadlines, precisionRequirements)

> projectsAI
  projectNames projDeadlines precisionRequirements
1  DeliveryMan    2021-09-30                180.00
2  WheresCroc     2021-09-30                  8.00
3  Diagnostics    2021-10-15                  0.99

# To refer to each vector (column) we can use its name preceded by $ letter
> projectsAI$precisionRequirements
[1] 180.00   8.00   0.99

# or even we can use the addressing format that we use for matrices
projectsAI[,3]
[1] 180.00   8.00   0.99

# To add a new row to a data frame we can use the (rbind) command
> projectsAI <- rbind(projectsAI,c("Image Analysis",projDeadlines = as.Date("20
    ↪ 21-10-30",format = "%Y-%m-%d"), precisionRequirements = 1))
> projectsAI
    projectNames projDeadlines precisionRequirements
1    DeliveryMan    2021-09-30                   180
2     WheresCroc    2021-09-30                     8
3    Diagnostics    2021-10-15                  0.99
4 Image Analysis    2021-10-30                     1

# the aggregate function is a useful tool to extract various information from
    ↪ dataframes. (You do not need it for the projects in this course). For
    ↪ example, to check how many projects have to be submitted per month in
    ↪ the AI course, you can use the following command:
> aggregate(x = projectsAI["projDeadlines"],
                      by = list(month = substr(projectsAI$projDeadlines, 1, 7))
                          ↪ , FUN = length)

    month projDeadlines
1 2021-09             2
2 2021-10             2
```

1.4 list

In R lists act as containers. This means that it can encompass any mixture of various data types and data structures with any lengths.

```
myList1 <- list(name = "AI course", period1 = TRUE, credits = 5, noStudents = 8
    ↪ )
```

```
> myList
$name
[1] "AI course"

$period1
[1] TRUE

$credits
[1] 5

$noStudents
[1] 8

myList2 <- list(name = "AI course", period1 = TRUE, credits = 5, noStudents = 8
    ↪ , studentsNames = c("Alfred", "Anna", "Oskar", "Emma", "Hannes", "Amalia
    ↪ ", "Malin", "Per"), projGroups = list(group1 = c("Alfred", "Anna"),
    ↪ group2 = c("Oskar", "Emma"), group3 = c("Hannes", "Amalia"), group4 = c(
    ↪ "Malin", "Per")))

> myList2
$name
[1] "AI course"

$period1
[1] TRUE

$credits
[1] 5

$noStudents
[1] 8

$studentsNames
[1] "Alfred" "Anna" "Oskar" "Emma" "Hannes" "Amalia" "Malin" "Per"

$projGroups
$projGroups$group1
[1] "Alfred" "Anna"

$projGroups$group2
[1] "Oskar" "Emma"

$projGroups$group3
[1] "Hannes" "Amalia"

$projGroups$group4
[1] "Malin" "Per"
```

As you can see, `myList1` and `myList2` contain multiple data types. Moreover, `myList2` encompasses another list (`projGroups`) that shows lists are powerful data structures that can maintain any mixture of data types and data structures. To refer to a specific element in a list we can use both the element's name (`$name`) and the element's index (inside a double bracket).

```
> myList2$studentsNames
[1] "Alfred" "Anna" "Oskar" "Emma" "Hannes" "Amalia" "Malin" "Per"

> myList2[[5]]
[1] "Alfred" "Anna" "Oskar" "Emma" "Hannes" "Amalia" "Malin" "Per"

> myList2$projGroups$group1
[1] "Alfred" "Anna"

> myList2[[6]][[1]]
[1] "Alfred" "Anna"
```

Suppose we have a set of objects that are located in our store. Each object has a value and the location of an object in the store is specified with a triple (corridor, partition, shelf). For example, object o has the value of v that is stored in *(c, p, s)*. The list that keep information about all objects is:

```
> myObject1 <- list(name = "Table", value = 1000, corridor = 3, partition = 5,
    ↪ shelf = 1)

> myObject2 <- list(name = "Chair", value = 500, corridor = 6, partition = 2,
    ↪ shelf = 2)

> myObject3 <- list(name = "Torch light", value = 1200, corridor = 10,
    ↪ partition = 1, shelf = 1)

> myObject4 <- list(name = "Vase", value = 100, corridor = 1, partition = 1,
    ↪ shelf = 5)

> allObjects <- list(myObject1, myObject2, myObject3, myObject4)
```

Suppose the cost of moving an object from the current store to the new one is a function of the location that that object is stored in: *cost = 100 × corridor + 20 × partition*. Our task is to sort the objects based on their value minus the cost of its movement, i.e., *value - cost*. To find this value for all objects, one way is using loops such as `for` and `while` to which we all are familiar in all programming languages. But please pay attention that generally these kinds of loops are slow in `R`. Better alternatives for loops in `R` are the `apply` family functions. The `apply` family function that is used for lists in `R` is `lapply`. The output of this command is also a list:

```
> sortingValues <- lapply(allObjects, function(item) item$value - 100*item$
    ↪ corridor - 20*item$partition)

> sortingValues
[[1]]
[1] 600

[[2]]
[1] -140

[[3]]
[1] 180

[[4]]
```

```
[1] -20
```

Therefore, we can move objects based on this order one by one from the current store to the new one.

```
> myIndex <- which.max(sortingValues)
> toMove <- allObjects[[myIndex]]

# removes that object from the current store:
> allObjects <- allObjects[[-myIndex]]
```

2 Functions

Defining functions in `R` is easy. You can define and store your functions in a file and use the `source()` command to invoke it into the `R`'s workspace. Command `ls()` shows the available variables and functions in your workspace. If you enter this command right now, I expect you do not have `myTutorialFunction()` available in your workspace. To define this function make a new file, let's call it `myFunction.R`.

```
# put the following function in file myFunction.R
myTutorialFunction <- function(a, b)
{
    if (is.numeric(a) & is.numeric(b))
    {
        return(a + b)
    }
    else
    {
        print("Please enter numeric values!")
    }
}

# to make this function available in workspace:
> source("/address to the file/myFunction.R")

> ls()
[1] "myTutorialFunction"

> myTutorialFunction(2, 3)
[1] 5
```

3 Using Functions in Packages

To start working on your assignments in this course, you need to install the packages that are available on the studium portal that are called `DeliveryMan`, `WheresCroc` and `Diagnostics`. To install the packages run the following command in an `R` session.

```
> install.packages("/address/DeliveryMan_1.1.0.tar.gz",repos = NULL, type="
    ↪ source")

> install.packages("/address/WheresCroc_1.2.2.tar.gz",repos = NULL, type="
```

```
    ↪ source")

> install.packages("/address/Diagnostics_1.2.0.tar.gz",repos = NULL, type="
    ↪ source")
```

To install other packages that are available on `cran-R`, you can easily run the following command:

```
> install.packages("package's name")
```

To use functions of a package, we have to make it available in the workspace with command `library()`:

```
> library(DeliveryMan)

# Now we can call the functions in the DeliveryMan package
> runDeliveryMan(carReady = basicDM,doPlot = TRUE)
```

If you do not make the installed package available in the workspace, whenever you want to call a fuction from that package you have to precede the package's name before:

```
# If you have not used the library(package's name) command
> DeliveryMan::runDeliveryMan(carReady = DeliveryMan::basicDM,doPlot = TRUE)
```

Please note that to call functions in a package that have been defined internal, we have to use the name of the package and triple colon (:::).

```
# This is an internal function
#' @keywords internal
myIntFunc <- function() {
  print("This is an internal function!")
}

> packageName:::myIntFunc()
[1] "This is an internal function!"
```

How to see the code of a function? To see a function's code you have two solutions: The first solution is to enter the function's name without parenthesis:

```
# To see the code of plotRoads() function in the DeliveryMan package
> DeliveryMan:::plotRoads
function (hroads, vroads)
{
    for (row in 1:nrow(hroads)) {
        for (col in 1:ncol(hroads)) {
            lines(c(row, row + 1), c(col, col), col = hroads[row, col])
    }
    ...
}
```

The second solution is to open the file that the functions' codes are in. For example, to see the implementation of the functions in the `DeliveryMan` package, check the `DM.R` file in the `R` folder in the `DeliveryMan_1.1.0.tar.gz` file.